

MNIST Handwritten Digit Classification - Notebook Code

MNIST Handwritten Digit Classification

This notebook demonstrates the model training and evaluation process.

1. Imports and Enhancements

First, we import the necessary libraries and define our data augmentation functions for digits '1' and '7'.

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
from scipy.ndimage import rotate, shift, affine_transform, center_of_mass
import joblib
import numpy as np
import json
import os

def add_crossbar(img):
    img = img.reshape(28, 28).copy()
    rows = np.any(img > 0.2, axis=1)
    if not np.any(rows): return img.flatten()
    rmin, rmax = np.where(rows)[0][[0, -1]]
    mid_row = (rmin + rmax) // 2

    cols = np.where(img[mid_row] > 0)[0]
    if len(cols) == 0:
        cols = np.where(np.any(img[mid_row-2:mid_row+3] > 0, axis=0))[0]

    if len(cols) > 0:
        cmin, cmax = cols[0], cols[-1]
        img[mid_row, max(0, cmin-3):min(28, cmax+4)] = 0.8
    return img.flatten()

def augment_digit(img, label):
    img = img.reshape(28, 28)
    choice = np.random.choice(['rotate', 'shift', 'shear', 'crossbar' if label == '7' else 'rotate'])

    if choice == 'rotate':
        return rotate(img, np.random.uniform(-15, 15), reshape=False).flatten()
    elif choice == 'shift':
        return shift(img, np.random.uniform(-3, 3, size=2)).flatten()
    elif choice == 'shear':
        af = np.array([[1, np.random.uniform(-0.3, 0.3), 0], [0, 1, 0]])
        return affine_transform(img, af[:, :2]).flatten()
    elif choice == 'crossbar':
        return add_crossbar(img)
    return img.flatten()
```

2. Load and Prepare the Dataset

MNIST Handwritten Digit Classification - Notebook Code

We fetch the dataset from OpenML and cache it locally.

```
data_cache = "mnist_data.joblib"
if os.path.exists(data_cache):
    print("Loading cached MNIST dataset...")
    X, y = joblib.load(data_cache)
else:
    print("Fetching MNIST dataset from OpenML...")
    X, y = fetch_openml('mnist_784', version=1, return_X_y=True, as_frame=False)
    X = X / 255.0
    joblib.dump((X, y), data_cache)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Data Augmentation

We apply precision augmentation to the '1' and '7' digits to make the model more robust.

```
print("Starting Final Precision Augmentation...")
X_aug_list = [X_train]
y_aug_list = [y_train]

for label in ['1', '7']:
    idx = np.where(y_train == label)[0]
    extra_x = [augment_digit(X_train[np.random.choice(idx)], label) for _ in range(20000)]
    X_aug_list.append(np.array(extra_x))
    y_aug_list.append(np.array([label]*20000))

X_final = np.concatenate(X_aug_list)
y_final = np.concatenate(y_aug_list)
print(f"Total training samples after augmentation: {len(X_final)}")
```

4. Train the Neural Network

Training the `MLPClassifier`.

```
print(f"Training Precision Model...")
mlp = MLPClassifier(hidden_layer_sizes=(256, 128, 64), max_iter=35, alpha=1e-4,
                    solver='adam', verbose=True, random_state=42)

mlp.fit(X_final, y_final)

print("Saving precision model and analytics...")
joblib.dump(mlp, "mnist_model.joblib")

replay_idx = np.random.choice(len(X_train), 500, replace=False)
joblib.dump({"X": X_train[replay_idx], "y": y_train[replay_idx]}, "replay_buffer.joblib")
```

5. Evaluation and Metrics Definition

Evaluate the model's accuracy.

```
y_pred = mlp.predict(X_test)
```

MNIST Handwritten Digit Classification - Notebook Code

```
accuracy = accuracy_score(y_test, y_pred)
with open("metrics.json", "w") as f:
    json.dump({"accuracy": accuracy * 100, "test_loss": float(mlp.loss_)}, f)

cm = confusion_matrix(y_test, y_pred, labels=[str(i) for i in range(10)])
np.save("confusion_matrix.npy", cm)

sample_indices = np.random.choice(len(X_test), 15, replace=False)
samples = []
for i in sample_indices:
    samples.append({
        "image": X_test[i].reshape(28, 28).tolist(),
        "target": int(y_test[i]),
        "pred": int(y_pred[i])
    })
with open("samples.json", "w") as f:
    json.dump(samples, f)

print(f"Final Test Accuracy: {accuracy * 100:.2f}%")
```